

Lesson 1: Algorithms Solve Problems

45 minutes

Overview

Students will complete two exploratory activities that introduce the concept of a problem and an algorithm. In the first students answer a series of questions about birthdates and names of their classmates. They then discuss the similarities and differences between the problems. In the second activity students are given six different algorithms and must analyze them to determine which they think are the same or different. At the end of the lesson they are introduced to the formal definitions of a problem and an algorithm.

Standards

Full Course Alignment

CSP Conceptual Framework

- **AAP-2** - The way statements are sequenced and combined in a program determines the computed result. Programs incorporate iteration and selection constructs to represent repetition and make decisions to handle varied input values.

Agenda

The Pre-Unit Pulse

Warm Up (5 minutes)

Activity (30 minutes)

Comparing Problems

Comparing Algorithms

Wrap Up (10 minutes)

Assessment: Check For Understanding

Objectives

Students will be able to:

- Explain that some algorithms may look or operate differently but still solve the same problem.
- Explain that some problems may look different but be similar or look similar but be different.
- Explain the formal definitions of a problem, an algorithm, sequencing, selection, and iteration.

Preparation

- Review the algorithms students will be comparing in the second activity to make sure you are prepared to support students in trying them out.
- Check the **"Teacher's Lounge"** forum for verified teachers to find additional strategies or resources shared by fellow teachers
- If you are teaching virtually, consider checking our **Virtual Lesson Modifications**

Links

Heads Up! Please make a copy of any documents you plan to share with students.

For the teachers

- **Algorithms (Unit Slides)** - Slides
▼ Make a Copy
- **Algorithms Solve Problems** - Slides
▼ Make a Copy

For the students

- **Unit 10 Journal (Digital)** - Resource
▼ Make a Copy
- **Unit 10 Journal (Interactive Notebook)** - Resource
▼ Make a Copy

Vocabulary

- **Algorithm** - a finite set of instructions that accomplish a task
- **Iteration** - a repetitive portion of an algorithm which repeats a specified number of times or until a given condition is met
- **Problem** - a general description of a task that can (or cannot) be solved with an algorithm
- **Selection** - deciding which steps to do next
- **Sequencing** - putting steps in an order

Teaching Guide

The Pre-Unit Pulse

Have your students complete these questions independently some time before starting this unit.

☰ 1-2

The Pre-Unit Pulse

1

2

💡 Teaching Tip

Pre-Unit Pulse Questions: The answers to these questions can provide insights into the preferences, strengths, and motivations of your students. If you'd like to adapt the questions or add in your own, you may want to make a copy of [this Google Form](#) which already has the existing questions populated.

For more tips and ideas of how to use these questions, check out the [CSP Guide to Pre-Unit Pulse Questions](#)

Warm Up (5 minutes)

Distribute: Give each student a copy of the Unit 10 Journal ([Digital/Interactive Notebook](#)) or have students set up this unit's section in their CSP journals.

☞ **Discuss:** *What makes two pieces of code "the same"? Could there ever be two pieces of code that you consider to be "the same" even if they aren't identical?*

Discussion Goal: This is an optional discussion. If you are able to move directly to the main activity you should do so. This discussion should get students thinking about the themes of the lesson.

There are no "wrong answers" here though you should expect answers that focus on the fact that often there's lots of ways to write code that does the same thing.

Have students think about their answers on their own, then share with a partner, and then finally discuss responses with the entire room.

Activity (30 minutes)

Remarks

Today we are going to kick off a short unit about how computer scientists think about problem solving. A really important skill will be recognizing patterns and similarities.

Comparing Problems

Distribute: Ask students to take out their journals or give them some blank paper for working on the following problems

Show the slides for the ten problems students will need to solve.

Circulate: Ask students to review the problems for one minute, and then let them move around the room collecting information needed to solve the problems. This may take them several minutes.

 **Discuss:** *Which problems did you need to do something similar in order to solve them?*

Discussion Goal: This discussion should focus on what made the problems that students solved similar to one another. You likely will want to put the problems back on the screen to make it easier to talk through. Here are some connections you may pull out though there are more students may make.

- Problems 1 and 2 are very similar. As soon as you find one person who meets the criteria you know you're done.
- Problem 3 and 4 are very similar but you need to talk to every other student to answer it. You only need to keep track of the closest birthday you've heard so far, however.
- Problem 5 is easy to solve as soon as you've solved problems 3 and 4.
- Problem 6 - 10 require you to have written down everyone's birthday, likely in order.
- Problems 7 - 9 are the same problem but for different numbers of people. Whatever strategy you use for one of those would be helpful to solve the others
- Problem 10 is a different version of problem 7 but instead of finding the smallest gap you're finding the largest.

Have students discuss the prompt with a neighbor before asking them to share with the room. Lead a discussion on their experiences.

Remarks

The first thing that we need to think about as computer scientists is what is a "problem". We just looked at 10 problems, but as we discussed, a lot of them are similar. If we solve one problem we may actually solve another, or at least have a good idea for how to start solving another. As computer scientists it's important to ask "have I seen this problem before" or "how is this problem similar to others I've solved?"

Comparing Algorithms

Remarks

We just thought about whether problems are similar. Now we're going to look at whether we're actually solving the same problem.

 **Discuss:** Which of these algorithms are “the same” as one another?

Discussion Goal: This discussion should focus on what made the algorithms the same. While they are designed to fall into two categories, ideally a number of points should come out of this discussion.

- Algorithms 1, 3, and 4 draw a square while 2, 5, and 6 draw a rectangle
- Algorithms 4 and 6 are written using a flowchart while 1, 2, 3, and 5 are written in the AP Pseudocode
- Some of these algorithms turn the robot right by turning left three times. It's debateable whether we can really call these algorithms "the same"
- Some of these algorithms create lists or variables to store information. Depending on the context we may not be able to call these algorithms "the same"

Circulate: Ask students to review the algorithms with a partner and group them into categories. Move around the room making sure students are not getting stuck. If they finish early push them to see if they can think about the problem in a different way.

 **Discuss:** Discuss with another group. which of these algorithms are “the same” as one another?

Have students discuss with another before asking them to share with the room. Lead a discussion on their experiences using tips from the discussion goal at the side.

Assessment Opportunity ▲

Create Performance Task Written Response Practice

In order to help prepare students for the new component of the AP Create Performance Task, there will be several opportunities throughout the curriculum to practice creating written responses aligned with the learning objectives that might be assessed on the exam.

Sample Prompt: *What makes two pieces of code “the same”? Could there ever be two pieces of code that you consider to be “the same” even if they aren’t identical?*

Unit Journal Template

Sample Response for Instructors



What makes two pieces of code “the same”? Could there ever be two pieces of code that you consider to be “the same” even if they aren’t identical?

In the box on the left, write your response. In the box on the right, find screenshots that represent why you chose your response, and then annotate the code.

Write your response here....

Find screenshots that represent why you chose your response, and then annotate the code

Wrap Up (10 minutes)

 **Journal:** Students add the following vocabulary words and definitions to their journals: problem, algorithm, sequencing, selection, iteration.

Discuss: *How did today’s activities change the way you think about algorithms and problems?*

Have students think about their answers on their own, then share with a partner, and then finally discuss responses with the entire room.

Discussion Goal: Use this discussion to reinforce vocabulary introduced in the slides and check in on whether students have begun the transition towards thinking on a higher level about algorithms and problems.

Remarks

Computer scientists don't just think about "code", they think about problems and the algorithms that solve them. In this unit we're going to explore what makes two problems, or two algorithms, similar or different from one another, and the way computer scientists talk about them. Not only will you be a better programmer, but you'll get to work on some really interesting problems along the way.

Assessment: Check For Understanding

Check For Understanding Question(s) and solutions can be found in each lesson on Code Studio. These questions can be used for an exit ticket.

Question: In your own words explain the difference between a problem and an algorithm.





This work is available under a **Creative Commons License (CC BY-NC-SA 4.0)**.

If you are interested in licensing CodeAI materials for commercial purposes **contact us**.